

使用 NumPy 实现两层 ReLU 网络

1 实验背景

根据通用近似定理我们可以知道，一个包含足够多神经元的单层前馈神经网络，如果使用非线性激活函数（例如 ReLU），可以在闭区间上以任意精度逼近任何连续函数。这次作业的目标是使用库函数或者 NumPy 手动实现反向传播算法，构建一个两层 ReLU 网络来拟合一个复合函数，以此验证神经网络对复杂函数的近似能力。我选择通过 NumPy 手动实现代码，不使用现有的 Python 第三方库，在完成作业的过程中，我对于梯度下降、反向传播等核心算法有了一定的学习和理解。

2 数据准备

2.1 函数定义

我设计了下面的函数作为拟合目标，这个函数可以增加拟合难度，并检验模型对高频成分的捕捉能力，：

$$f(x) = 2 \sin(x) + 0.5 \cos(3x) + 0.2x, \quad x \in [-5, 5]. \quad (1)$$

该函数在定义域内包含多个波峰与波谷，兼具线性和非线性特征，能够有效测试模型的局部拟合与泛化性能。

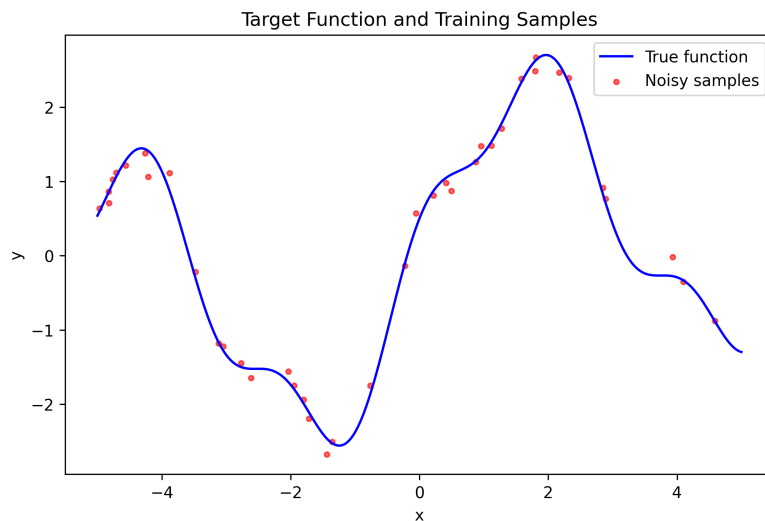


图 1: 目标函数及训练样本（每 20 个点显示一个）

2.2 数据采集

- 采样区间：函数的定义域为 $x \in [-5, 5]$ ，在代码中，我在这个定义域内使用均匀采样的方法采样 1000 个点。
- 数据集划分：我先对数据集进行随机打乱，然后按照 8:2 的比例分割为训练集（800 个样本）和测试集（200 个样本）。
- 噪声添加：最后，为了模拟真实数据，我向训练集标签添加标准差为 0.1 的高斯噪声。

图 1 展示了无噪声的真实函数曲线以及带噪声的训练样本分布。

3 模型描述

3.1 模型架构

我采用单层全连接神经网络：

- 输入层：1 个神经元，对应自变量 x 。
- 隐藏层：1024 个神经元，激活函数为 ReLU： $a = \max(0, z)$ 。
- 输出层：1 个线性神经元，输出拟合值 $\hat{y}$ 。

3.2 核心算法实现

- **参数初始化**：我采用 He 初始化，权重从正态分布 $\mathcal{N}(0, \sqrt{2/n_{\text{in}}})$ 中采样，偏置初始化为 0，可以有效避免梯度消失或爆炸。
- **损失函数**：均方误差（MSE）

$$L = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2.$$

- **梯度计算**：手动推导反向传播，关键公式如下：

$$\begin{aligned} \frac{\partial L}{\partial z_2} &= \frac{2}{m}(\hat{y} - y), \\ \frac{\partial L}{\partial W_2} &= a_1^\top \frac{\partial L}{\partial z_2}, \quad \frac{\partial L}{\partial b_2} = \sum \frac{\partial L}{\partial z_2}, \\ \frac{\partial L}{\partial a_1} &= \frac{\partial L}{\partial z_2} W_2^\top, \\ \frac{\partial L}{\partial z_1} &= \frac{\partial L}{\partial a_1} \odot \mathbf{1}_{z_1 > 0}, \\ \frac{\partial L}{\partial W_1} &= x^\top \frac{\partial L}{\partial z_1}, \quad \frac{\partial L}{\partial b_1} = \sum \frac{\partial L}{\partial z_1}. \end{aligned}$$

- **优化器**：动量梯度下降，动量因子 $\gamma = 0.9$ ，学习率 $\eta = 0.002$ 。更新规则为

$$v_t = \gamma v_{t-1} - \eta \nabla W, \quad W_{t+1} = W_t + v_t.$$

- **梯度裁剪**：将每个梯度的值裁剪至 $[-1, 1]$ 区间，防止梯度爆炸。
- **训练超参数**：迭代次数 $epochs = 30000$ ，每次迭代使用全部训练样本（全批量梯度下降）。

4 拟合效果

4.1 训练损失曲线

图 2 记录了训练过程中 MSE 随迭代次数的变化。前 5000 个 epoch 损失下降迅速，之后逐渐收敛，最终训练损失稳定在约 0.24 附近，表明模型已经较好地拟合了训练数据。

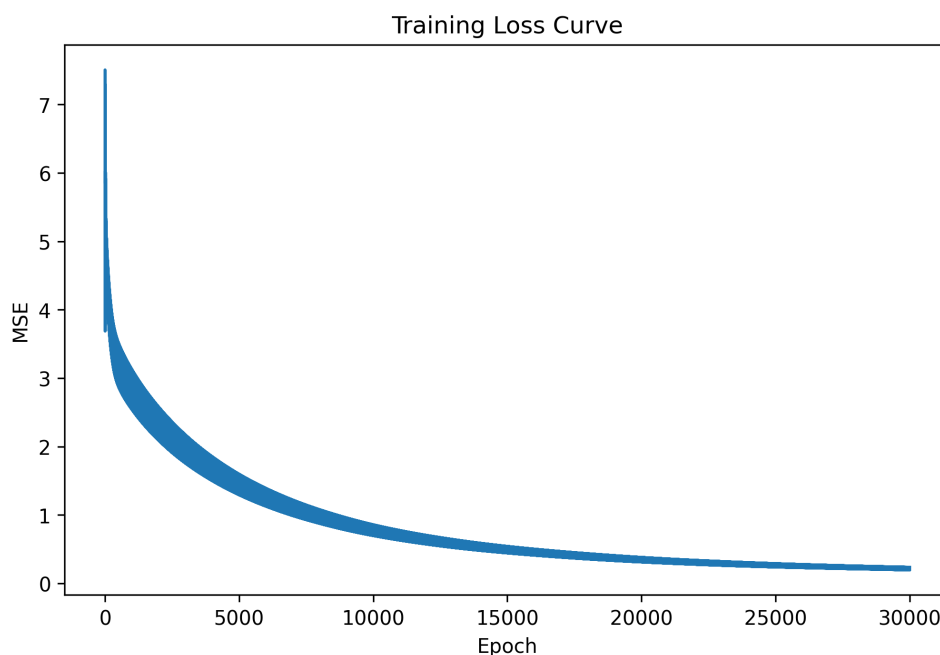


图 2: 训练过程中的 MSE 损失曲线

4.2 拟合结果可视化

图 3 展示了模型在测试区间上的预测结果。蓝色实线为真实函数，红色虚线为网络预测值，黑色散点为测试样本。可见模型在区间中部 $(-3, 3)$ 范围内拟合效果极佳，能够准确捕捉函数的振荡细节。但在区间边缘 $x \approx -5$ 和 $x \approx 5$ 处，预测曲线呈现明显的直线趋势，这是由于 ReLU 网络本质上是分段线性函数，在训练数据覆盖的区域之外，网络会保持最后一个线性片段的斜率进行外推，导致拟合能力受限。

后续可以对隐藏层维度和训练回合数进行调整，从而获得更好的效果。

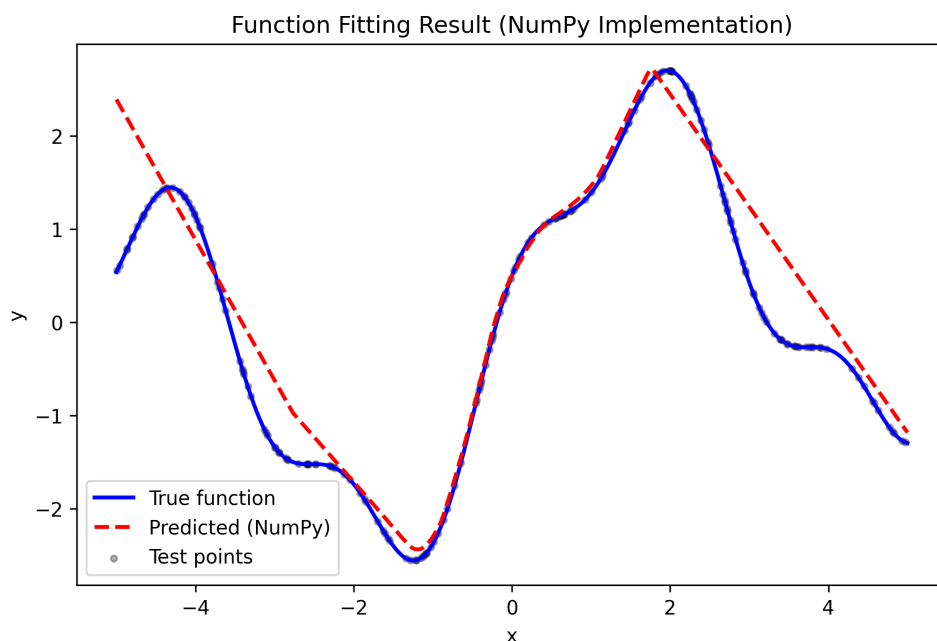


图 3: 测试集上的拟合效果对比

4.3 评估结果

在测试集上计算均方误差：

$$\text{Test MSE} = 0.217928.$$

该误差值较小，考虑到目标函数复杂度以及添加的噪声水平，说明模型具有良好的泛化能力。

5 结论

在本次实验中，我通过 NumPy 手动实现了一个两层的 ReLU 网络，并且拟合了一个包含正弦、余弦和线性成分的复合函数。从训练过程和实验结果中我有一些发现。

1. 首先是单层 ReLU 网络在隐藏层神经元足够多时（1024 个）能够以较高精度逼近复杂连续函数，这也验证了通用近似定理。
2. 引入 He 初始化、动量梯度下降和梯度裁剪等技术，可以比较好地提升训练的稳定性与收敛速度。
3. 虽然网络在训练数据覆盖的区域内表现优异，但在区域边缘之外的能力有限，呈现分段线性特性，这是由 ReLU 激活函数本身的数学性质决定的。后续可以针对网络的隐藏层和训练回合数进行一些调整，或许可以获得更好的效果。

通过本次实验，加深了我对于反向传播算法的理解，我也认识到手动实现神经网络代码是十分重要的。后续我也会继续学习深度学习领域的知识，提高个人能力。